

API handlers and mutations

Module 4 - Lesson 2

Overview

Next.js route handlers let you define API endpoints inside your app. Mutations update the database in response to user actions, and revalidation keeps the UI fresh.

Key Concepts

- Route handlers are files named `route.ts` that export `GET`, `POST`, `PUT`, `DELETE` functions.
- They receive a `Request` and return a `Response` or `NextResponse.json()`.
- Validate input with a schema library before touching the database.
- Mutations in Server Actions or API routes run server-side where secrets stay safe.
- `revalidatePath/revalidateTag` invalidate cached data so the UI reflects the mutation.
- Follow REST conventions and return meaningful status codes and error shapes.

Example

```
export async function POST(req: Request) {
  const body = await req.json();
  const parsed = schema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json({ error: parsed.error }, { status: 400 });
  }
  const lesson = await db.insert(lessons).values(parsed.data).returning();
  revalidatePath('/lessons');
  return NextResponse.json(lesson[0], { status: 201 });
}
```

Summary

Route handlers give you real API endpoints in your Next.js app. Validating input, writing mutations server-side, and revalidating caches keeps data correct and pages current.

Practice Checklist

- Create a route handler that creates a resource with validation.
- Wire a form submission to the handler and update the UI after success.
- Add revalidation so list pages reflect newly created data.